



Sesión de Laboratorio 10:

# SKEME PROTOCOL



## SELECTED TOPICS OF CRYPTOGRAPHY

Profesora || Sandra Díaz Santiago  
Grupo || 7CM1  
Alumno || Areli Alejandra Guevara Badillo

14 de junio de 2026

## ATIVIDADES DE PROGRAMACIÓN

El sistema está dividido en tres componentes lógicos:

1. Un módulo criptográfico (crypto\_utils.py).
2. Un nodo Iniciador/Cliente (client.py).
3. Un nodo Respondedor/Servidor (server.py).

### Módulo Criptográfico (crypto\_utils.py)

Este archivo centraliza todas las operaciones criptográficas requeridas por el protocolo. Se utilizó la biblioteca cryptography.hazmat de Python, estándar en la industria para instanciar primitivas de forma segura y auditable. A continuación, se detallan los componentes implementados:

| Función                            | Algoritmo      | Propósito en SKEME                                                     | Fragmento clave                                                                         |
|------------------------------------|----------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>generate_keypair()</b>          | RSA-2048       | Genera par de llaves de identidad para Iniciador (I) y Respondedor (R) | <code>rsa.generate_private_key(public_exponent=65537, key_size=2048)</code>             |
| <b>encrypt() / decrypt()</b>       | RSA-OAEP       | Transporte seguro de identificadores y nonces en FASE 1                | <code>padding.OAEP(mgf=padding.MGF1(hashes.SHA256()), algorithm=hashes.SHA256())</code> |
| <b>generate_dh_parameters()</b>    | Diffie-Hellman | Parámetros públicos p y g para intercambio efímero                     | <code>dh.generate_parameters(generator=2, key_size=512)</code>                          |
| <b>generate_dh_keypair()</b>       | DH efímero     | Genera llaves temporales x/X y y/Y para FASE 2                         | <code>parameters.generate_private_key()</code>                                          |
| <b>compute_hash()</b>              | SHA-256        | Derivación de k_mac y k_sess (KDF simplificado)                        | <code>hashes.Hash(hashes.SHA256())</code>                                               |
| <b>compute_hmac()</b>              | HMAC-SHA256    | Autenticación mutua en FASE 3                                          | <code>hmac.HMAC(key, hashes.SHA256())</code>                                            |
| <b>serialize_* / deserialize_*</b> | PEM            | Codificación segura para transmisión por sockets.                      | <code>public_bytes(encoding=Encoding.PEM, ...)</code>                                   |

## Flujo del Protocolo SKEME Implementado

La simulación sigue la estructura clásica de SKEME, dividiéndose en 4 fases bien diferenciadas:

| Iniciador (Client)                             | Respondedor (Server) |
|------------------------------------------------|----------------------|
|                                                |                      |
| ----- FASE 0: Setup Inicial -----              |                      |
| <----- pk_R (RSA Pub), parámetros DH -----     |                      |
| ----- pk_I (RSA Pub) ----->                    |                      |
|                                                |                      |
| ----- FASE 1: SHARE -----                      |                      |
| ----- c_I = RSA-OAEP(pk_R, ID_I    k_I) -----> |                      |
| <----- c_R = RSA-OAEP(pk_I, k_R) -----         |                      |
| k_mac = SHA256(k_I    k_R)                     |                      |
|                                                |                      |
| ----- FASE 2: EXCH (DHE) -----                 |                      |
| ----- X (pub DH efímero de I) ----->           |                      |
| <----- Y (pub DH efímero de R) -----           |                      |
|                                                |                      |
| ----- FASE 3: AUTH & KDF -----                 |                      |
| ----- mac_I = HMAC(k_mac, Y  X  ID_I  ID_R)->  |                      |
| <----- mac_R = HMAC(k_mac, X  Y  ID_R  ID_I)-- |                      |
| k_sess = SHA256(DH_shared_secret)              |                      |
|                                                |                      |
|                                                |                      |

## DESCRIPCIÓN FASE POR FASE CON CÓDIGO

### FASE 0: Distribución de Llaves y Parámetros

Se intercambian las llaves públicas RSA estáticas y los parámetros DH. Esto establece la identidad criptográfica y prepara el terreno para el intercambio efímero.

```
1 # client.py
2 pk_R_bytes = s.recv(4096)
3 pk_R = deserialize_pubkey(pk_R_bytes)
4 dh_params_bytes = s.recv(4096)
5 dh_params = deserialize_dh_params(dh_params_bytes)
6 s.sendall(serialize_pubkey(pk_I))
```

**FASE 1: SHARE** (Transporte de Identificadores)

Cada parte genera un nonce/identificador aleatorio de 16 bytes ( $k_I$ ,  $k_R$ ). Se cifran con la llave pública del par y se intercambian. Ambos derivan una **llave de autenticación**  $k_{mac}$ .

```
1 # client.py
2 k_I = os.urandom(16)
3 payload_I = ID_I + k_I
4 c_I = encrypt(pk_R, payload_I)
5 s.sendall(c_I)
6
7 c_R = s.recv(4096)
8 k_R = decrypt(sk_I, c_R)
9 k_mac = compute_hash(k_I + k_R)  # Derivación de clave de autenticación
```

**FASE 2: EXCH** (Intercambio Diffie-Hellman Efímero)

Se genera un par de llaves DH temporal por cada parte. El intercambio de las llaves públicas X e Y permite calcular un **secreto compartido** sin transmitirlo por la red.

```
1 # server.py
2 X_bytes = conn.recv(4096)
3 y_sk, Y_pk = generate_dh_keypair(dh_params)
4 Y_bytes = serialize_pubkey(Y_pk)
5 conn.sendall(Y_bytes)
```

**FASE 3: AUTH & KDF** (Autenticación Mutua y Derivación de Sesión)

1. **Autenticación:** Cada parte firma un HMAC que incluye las llaves DH intercambiadas y las identidades. El orden de concatenación garantiza que  $mac_I \neq mac_R$  y previene ataques de reflexión.
2. **KDF:** Se calcula la clave de sesión  $k_{sess}$  aplicando SHA-256 al secreto DH.

```
1 # client.py
2 mac_I_payload = Y_bytes + X_bytes + ID_I + ID_R
3 mac_I = compute_hmac(k_mac, mac_I_payload)
4 s.sendall(mac_I)
5
6 mac_R = s.recv(4096)
7 # Verificación automática implícita en la lógica del servidor
8 shared_secret = x_sk.exchange(Y_pk)
9 k_sess = compute_hash(shared_secret)
```

*La implementación cumple con el requisito de instanciación explícita al utilizar las primitivas de bajo nivel cryptography.hazmat. Cada algoritmo (RSA, DH, SHA-256 y HMAC) se instancia manualmente definiendo sus parámetros de seguridad (tamaño de llave, generador, esquema de padding y función hash), evitando abstracciones que oculten el flujo criptográfico. Esta aproximación garantiza trazabilidad, facilita la auditoría académica y refleja las buenas prácticas de desarrollo seguro vistas en clase.*